Conceptual Architecture Analysis:

Gemini CLI

Group 9
The Unemployed

Video Link

Team Members

Kesiya Jacob
Group Leader - Abstract, Intro/ Overview, Low level divergences

Gabrielle Garey
Presenter - Low level conceptual, low level concrete archiecture

Sophia Bushareb
High level divergences, dependency graph, use cases

Grisha Tyukin
AI collaboration, high level descriptions, limitations

Ryan Walsh
Presenter - Sequence diagrams, conclusions, lessons learnt, AI collaboration

Calvin Wong
Data dictionary, naming conventions

Derivation Process

Discussions with our TAs, decision to incorporate parts of group 8's conceptual architecture

Examining Source code and dependency graphs using Understand Tool

Assigned code files to conceptual architecture components using our AI teammate and extensive research to create concrete architecture (box-and-arrows diagram)

Identified and created conceptual and concrete architectures for Tool Execution Framework as our subsystem of choice

Performed reflexion analysis on high-level and second-level architectures

Created sequence diagrams for use cases based on our concrete architecture

Dictionary & Naming Conventions

- **Architecture Styles:** Software styles that describe the behaviour and relationships of components in an application.
- **MCP:** Model Context Protocol, a standardized way for a language model to receive, interpret, and use contextual information from a system or application.
- **Server:** A computer that provides services, data, or resources for clients over a network.
- **NodeJS:** Open source Javascript runtime environment that allows Javascript to run outside of the browser.
- **LLM:** Large Language Model, generally a way to refer to AI.
- **API:** Application programming interface, a set of functions and procedures allowing the creation of applications that access the features or data of an operating system, application, or other service.
- **GUI:** Graphic user interface, a system that allows users to interact with digital devices through visual elements like icons, windows, menus, and pointers, rather than by typing-text based commands.
- **CLI:** Command line interface, terminal through which the user interacts with the program at a lower level. Alternative to GUI.
- **Roadmap:** A project board linked on the Github that visualizes the weekly division of tasks as a byproduct of the developer meetings hosted on the Discord server. Tasks are organized into categories, such as new features, improvements, and backlog items.
- **Github:** Developer platform allowing developers to store, manage, and share code. Provides access control, bug tracking, feature requests, and other project resources.

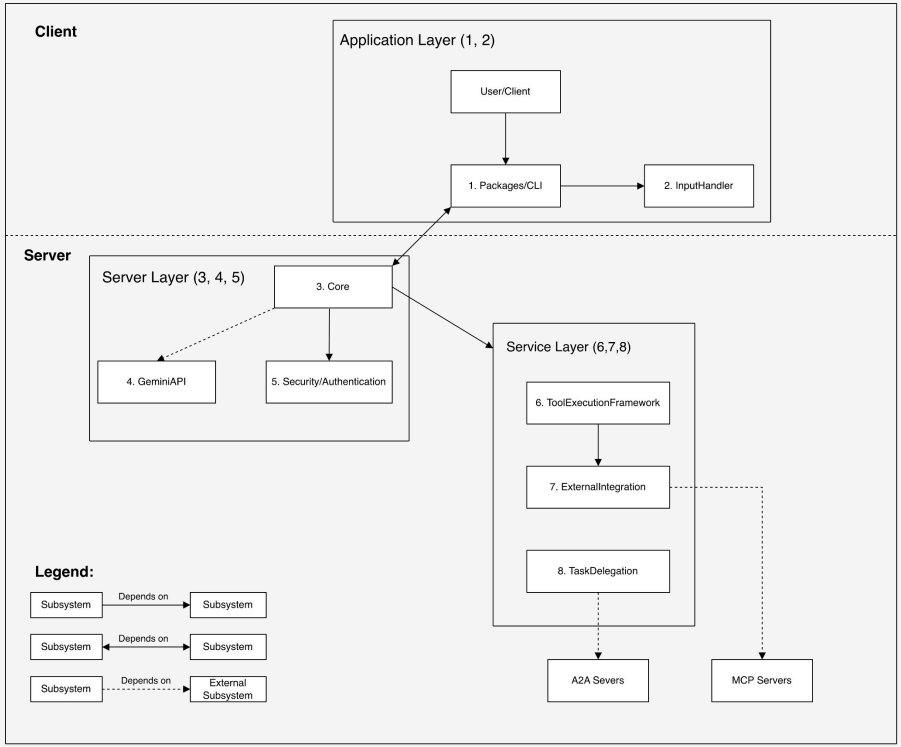
- **GUI:** Graphic User Interface
- **LLM:** Large Language Model
- **AI:** Artificial intelligence
- **API:** Application Programming Interface
- **OS:** Operating System
- **CLI:** Command line interface

High-Level Architecture

CONCEPTUAL ARCHITECTURE

Modified From Previous Conceptual Architecture

- Based on feedback and discussion with our TA
- Better representation of Gemini CLI architecture at the top-level
- Incorporated partial Layered style



CONCEPTUAL ARCHITECTURE: Subsystem Breakdown

Packages / CLI:

- UI of Gemini CLI; handles command line execution and user interaction

Input Handler:

- Parses and Validates commands, formats requests for processing

Core Subsystem:

- Central Controller of the architecture, coordinates workflow and routes requests

Gemini API:

- External LLM service, provides AI-generated responses & model-based processing

Security / Authentication:

- Handles Authentication and policy enforcement

Tool Execution Framework:

- Handles local tool execution, supports file modification or creation operations

External Integration:

- Connects to external MCP servers giving access to external services and tools

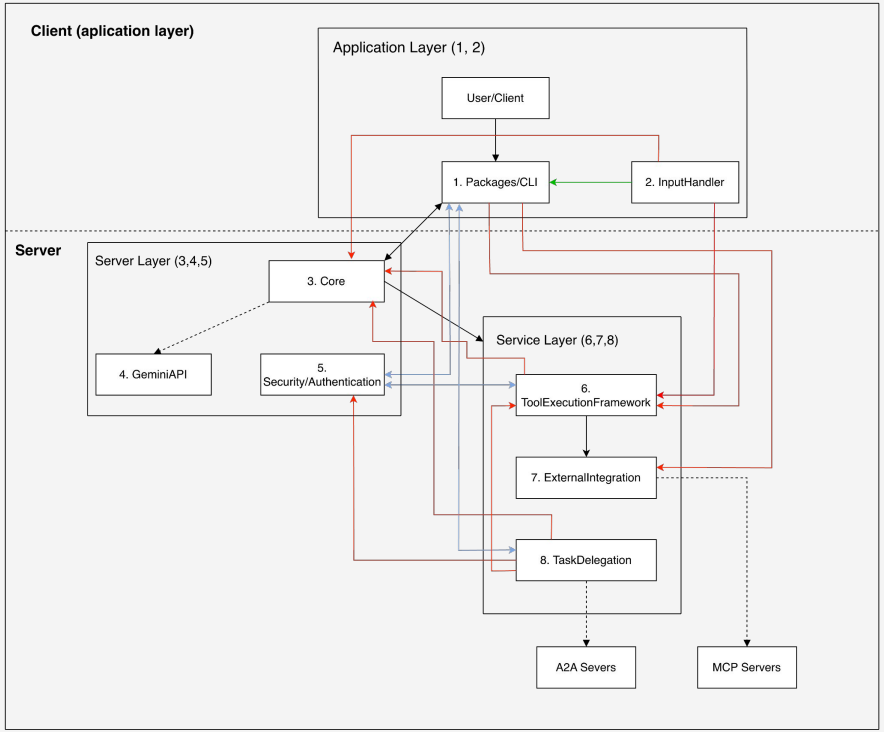
Task Delegation:

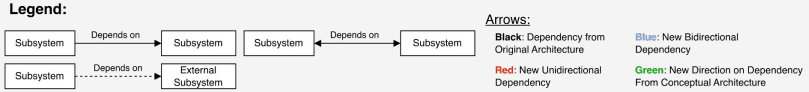
- Delegates tasks to external A2A servers, enabling distributed and scalable processing

Integration Endpoints:

- External tool providers connected via the Model Context Protocol
- Remote peer agents reachable via Google's Agent-to-Agent protocol

CONCRETE ARCHITECTURE





Built using Understand

- Layered style, in addition to c/s
- No new subsystems
- New dependencies
- New directions on previously identified dependencies

CONCRETE ARCHITECTURE: Subsystem Breakdown

Packages / CLI:

- Handles command line interaction, serves as entry point between front-end and back-end

Input Handler:

- Parses and Validates commands, formats requests for processing

Core Subsystem:

- Central controller, coordinates all high-level subsystem interactions.

Gemini API:

- Connects system to the external Gemini LLM, Supports AI-generated responses and model-based request processing

Security / Authentication:

- Enforces permissions, authentication, and policy checks

Tool Execution Framework:

- Manages execution of local tools such as shell commands and file operations, main execution component within the service layer

External Integration:

- Connects to external MCP servers giving access to external services and tools

Task Delegation:

- Delegates tasks to external A2A servers, enabling distributed and scalable processing

Integration Endpoints:

- External tool providers connected via the Model Context Protocol
- Remote peer agents reachable via Google's Agent-to-Agent protocol

CONCRETE ARCHITECTURE: Reflexion Analysis

Input Handler → Packages/CLI:

- skillUtils.ts imports SettingScope (a config scope) from settings.js to format user facing feedback labels for skill/ enable/disable actions, reversing the expected dependency direction.

Tool Execution Framework → Packages/Core:

- skillLoader.ts imports debugLogger (a shared logging utility) from Core to report runtime errors during skill file loading

Task Delegation → ToolExecutionFramework:

- agentLoader.ts imports isValidToolName from tool-names.ts to validate that tool names declared in an agent's definition file actually exist in the system at load time. Rather than TaskDelegation maintaining its own notion of valid tools, it reaches directly into ToolExecutionFramework's tool registry logic to perform this check, creating a cross-subsystem dependency

Task Delegation → Core:

- agentLoader.ts imports getErrorMessage from error.ts to convert raw error objects into readable descriptions when agent definition files fail to load or parse.

Second-Level Architecture: Tool Execution Framework

CONCEPTUAL ARCHITECTURE: Tool Execution Framework

Tool Dispatch:

- Receives tool calls from the Orchestration Engine and routing them through the framework

Permission Controller:

- Validates tool calls before execution, ensuring only authorized actions proceed

Tool Registry:

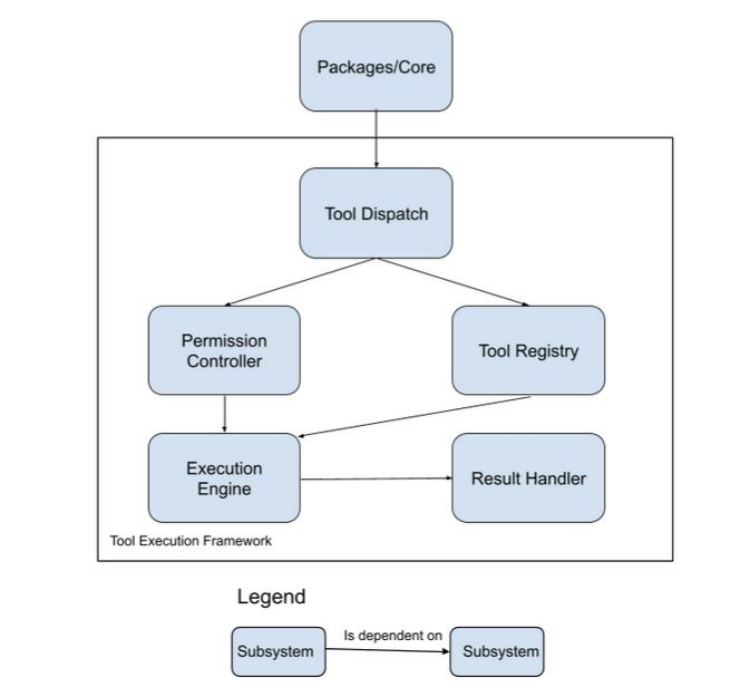
- Maintains the catalog of available tools

Execution Engine:

- Sets up the environment and runs tool after receiving confirmation from permission controller

Result Handler:

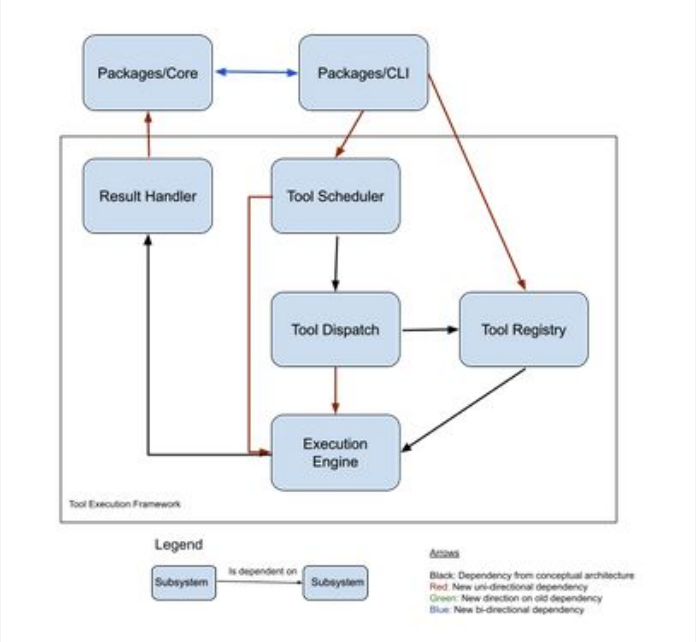
- Processes and formats the output from the Execution Engine before returning it upstream



Second-Level Conceptual Architecture of the Tool Execution Framework Subsystem

CONCRETE ARCHITECTURE

- New Components: Tool Scheduler
- There are new dependencies
  - Result Handler returns data to both front end and back end
  - Permission Controller is a method distributed across each tool's BaseTool implementation.
  - Overlooked that tool registry updates Orchestration Engine with catalog of available tools



Second-Level Concrete Architecture of the Tool Execution Framework Subsystem

REFLEXION ANALYSIS - Second-Level

Tool Dispatch → Execution Engine:

- scheduler.ts imports ToolConfirmationOutcome and AnyDeclarativeTool from tool.ts. This bypasses abstraction to support outcome assertions and facilitates mock tool instantiation.

Tool Scheduler → Execution Engine:

- coreToolScheduler.ts pulls core execution layer components into the scheduling logic. It imports five types/enums from tools.ts, specifically for managing ToolConfirmationOutcome and ToolConfirmationPayload to control UI-driven execution flows..

Packages/CLI → Packages/core:

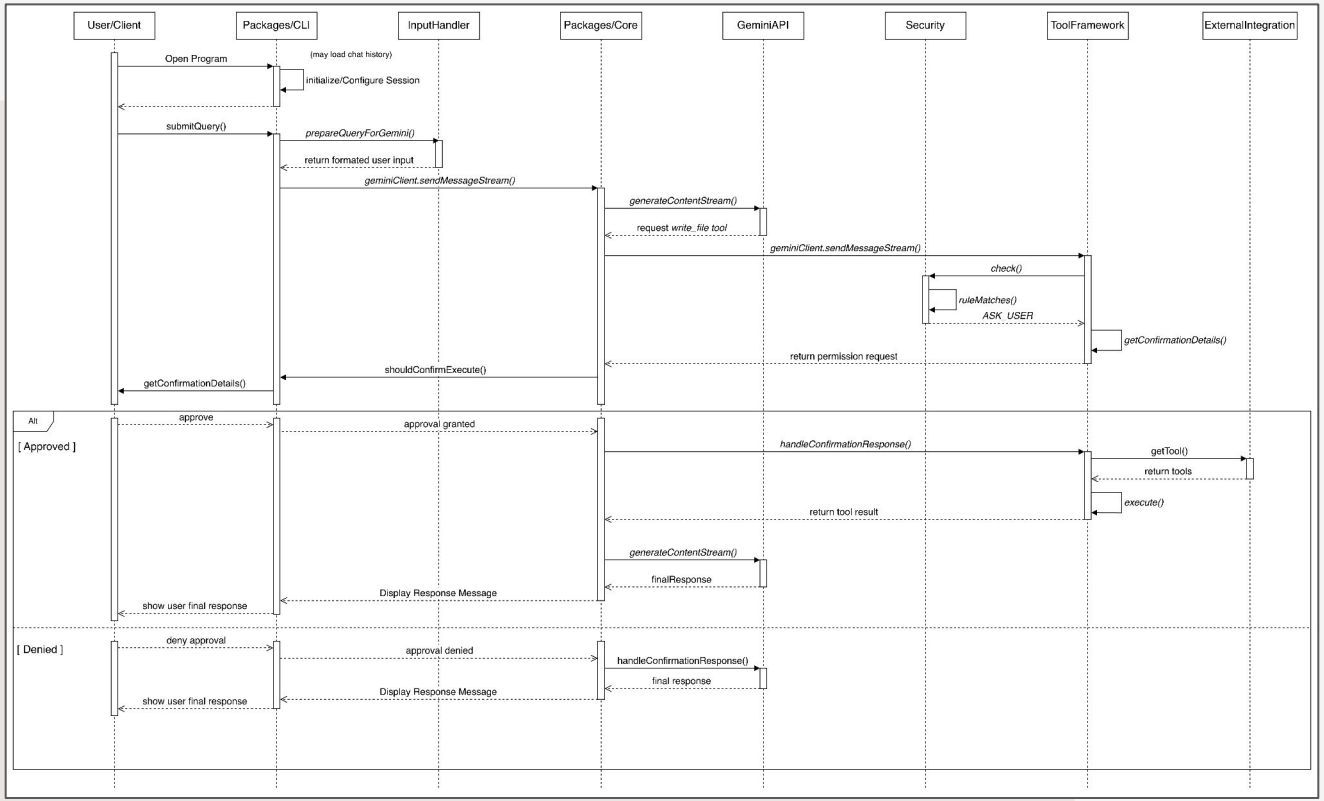
- The CLI layer reaches directly into Core's internal configuration by importing Config. This allows the CLI to instantiate and verify runtime environments for tool execution, bypassing a centralized orchestration interface.

Packages/CLI → Tool Registry:

- The CLI depends on low-level LLM response types (e.g., OllamaModelResponse, GeminiLLMChatMessage). These are required by VoidModelService to convert registry data into LLM-compatible formats

Sequence Diagram 1: Interactive

Use Case 1: A user types a prompt requesting file creation or modification



LIMITATIONS & LESSONS LEARNED

Diagram Readability

Misconceptions can arise from leaving out simple things like legends

Deciphering Scope of components

Deciphering the scope of components while test files include additional (separate) dependencies proved difficult.

Willingness To Redo previous work

A willingness to overhaul previous work, even if it is time consuming, can be very important in ensuring accuracy and quality.

Our AI Teammate: **perplexity**

Tasks Assigned

- Flesh out ideas on architectural style and subsystems
- Summarize and find sources

Interaction Protocol & Workflow

- Collaborate within partners to craft queries
- Use sero shot and 1-shot prompts
- Refine AI output by asking for specifics

AI agent
January 2026 version

perplexity

Quantitative impact: 12% of final deliverable

Validation & Quality Control

- Ask perplexity for sources
- Fact-check & cross-reference source material

Human-AI Team Dynamic

- Easy to collaborate and get efficient work done
- Importance of verifying AI-generated ideas

Conclusion

REFERENCES

- Dobaj, M., Thomas, S., Batala, B., Mirchandani, E., Mourad, S., & Kodukula, N. (2025, October 10). *A1: Conceptual Architecture of Void*. https://babinson2005.github.io/CISC322/assets/deliverables/A1-Conceptual%20Architecture%20Report-Group%2022.pdf
- microsoft. (2025, November 7). *GitHub - microsoft/vscode: Visual Studio Code*. GitHub.

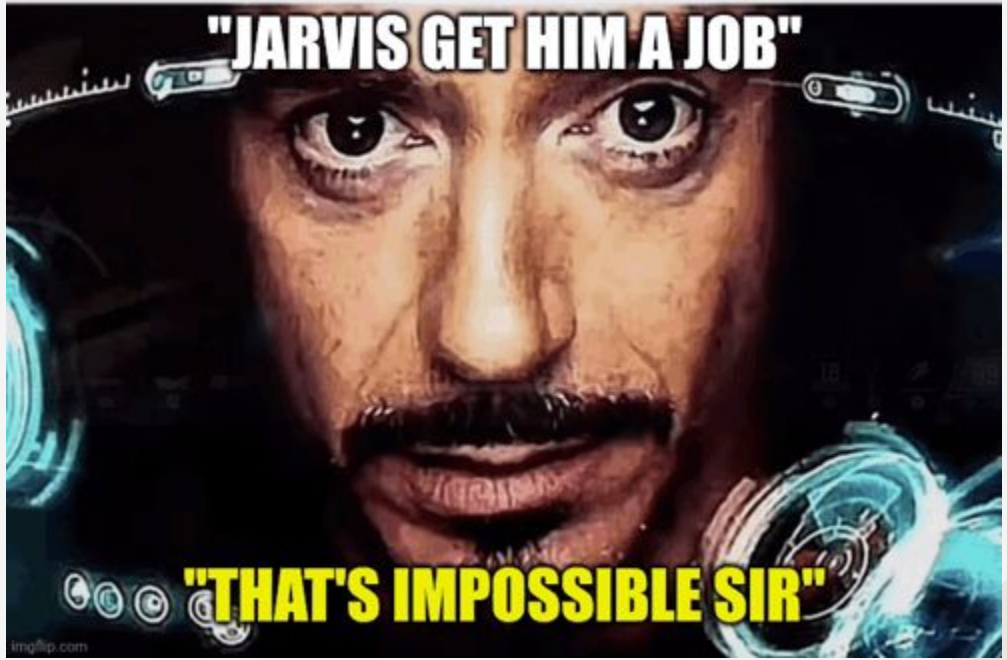
  https://github.com/microsoft/vscode

- Smith, C., Goncalves, E., Hunter, L., Feng, L., Bajwa, N., & Liu, S. (2025, October 10). *Conceptual Architecture Analysis of Void Editor*.

  https://docs.google.com/document/d/1Elojzi9N9sEBeqI3q3dOWE1T1NF9KReLTuI23DGBCd8/edit?tab=t.kvl7vvhulbj2

- voideditor. (2025, August 7). *GitHub - voidEditor/void*. GitHub. https://github.com/void/

Thanks for watching!



- The Unemployed*